

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAXPAROLA 30
#define MAXRIGA 80
```

```
int main(int argc, char *argv[])
{
    int freq[MAXPAROLA]; /* vettore di contatori
delle frequenze delle lunghezze delle parole */
    char riga[MAXRIGA];
    int i, inizio, lunghezza;
    FILE *f;
```

```
for(i=0; i<MAXPAROLA; i++)
    freq[i]=0;
```

```
if(argc != 2)
```

```
{
    fprintf(stderr, "ERRORE, serve un parametro con il nome del file\n");
    exit(1);
}
```

```
f = fopen(argv[1], "r");
if(f==NULL)
```

```
{
    fprintf(stderr, "ERRORE, impossibile aprire il file %s\n", argv[1]);
    exit(1);
}
```

```
while( fgets( riga, MAXRIGA, f ) != NULL )
```



Iter-Process Communication

UNIX IPC

Stefano Quer

Dipartimento di Automatica e Informatica

Politecnico di Torino

License Information

This work is licensed under the license



Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

① **BY:** Credit must be given to you, the creator.

② **NC:** Only noncommercial use of your work is permitted.

Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

③ **ND:** No derivatives or adaptations of your work are permitted.

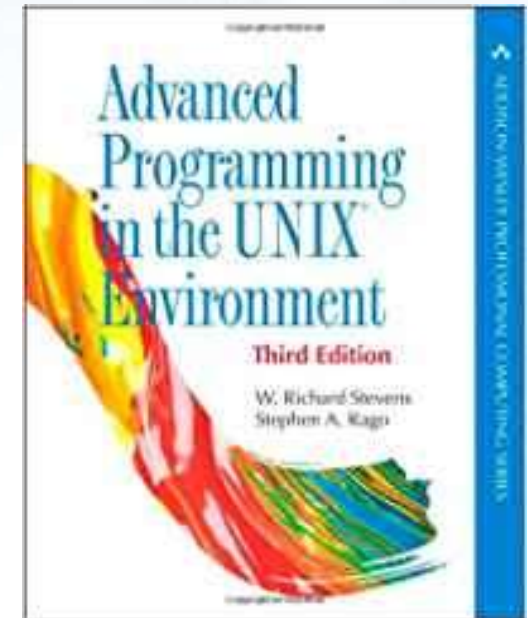
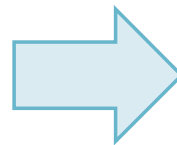
To view a copy of the license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1>

Premises

❖ Where are we?

- u01-courseIntroduction
- u02-review
- u03-cppBasics
- u04-cppLibrary
- u05-multithreading
- u06-synchronization
- u07-advancedIO
- u08-IPC

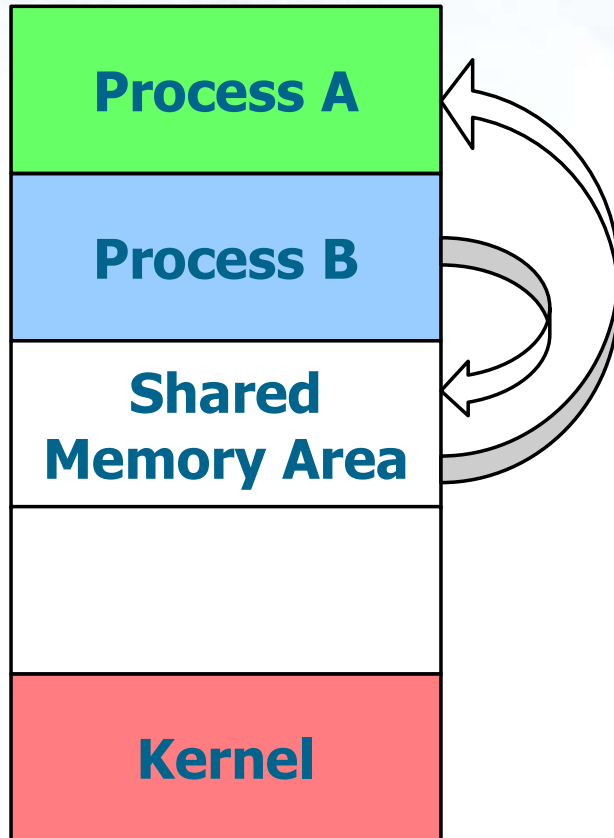


- u08s03e
- u08s05e
- u08s01-introduction.pdf
- u08s02-FIFOs.pdf
- u08s03-messageQueues.pdf
- u08s04-sharedMemory.pdf
- u08s05-shockets.pdf

Introduction

- ❖ Processes do not share their address space
 - Copy-on-Write duplicates the memory only when strictly required
- ❖ Information sharing among processes is referred to as **IPC** or **I**nter **P**rocess **C**ommunication
 - Processes may need to share information when running
 - On the same machine, **intra**-machine communication
 - On different machines, **inter**-machine communication
 - The communication models can be based on
 - Message exchange
 - Shared memory

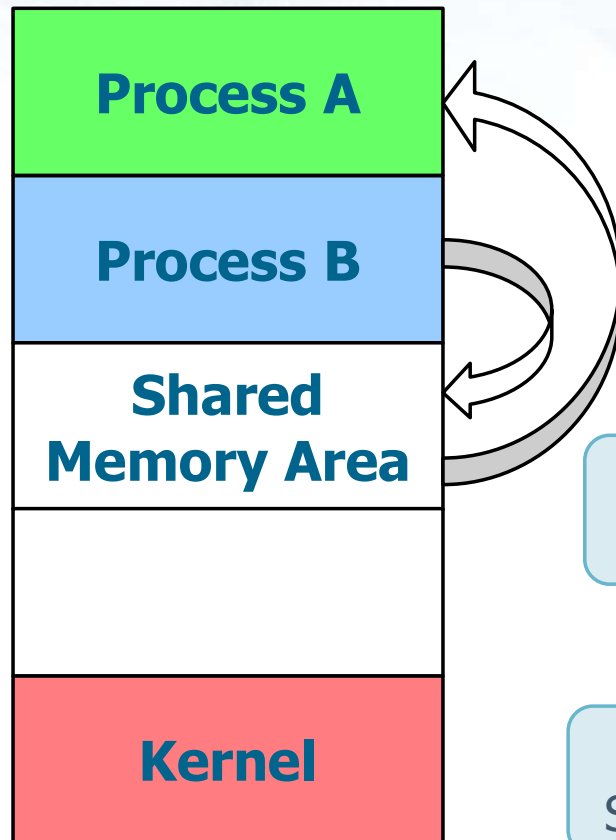
Communication models



❖ Shared memory

- Based on sharing a memory area and R/W data in this area
- Used for sharing a large amount of data

Communication models



❖ Shared memory

➤ Most common methods

▪ File sharing

- Sharing the name or the file pointer or descriptor before fork/exec

Course of OS

▪ File mapping

- A file is mapped into the address process space

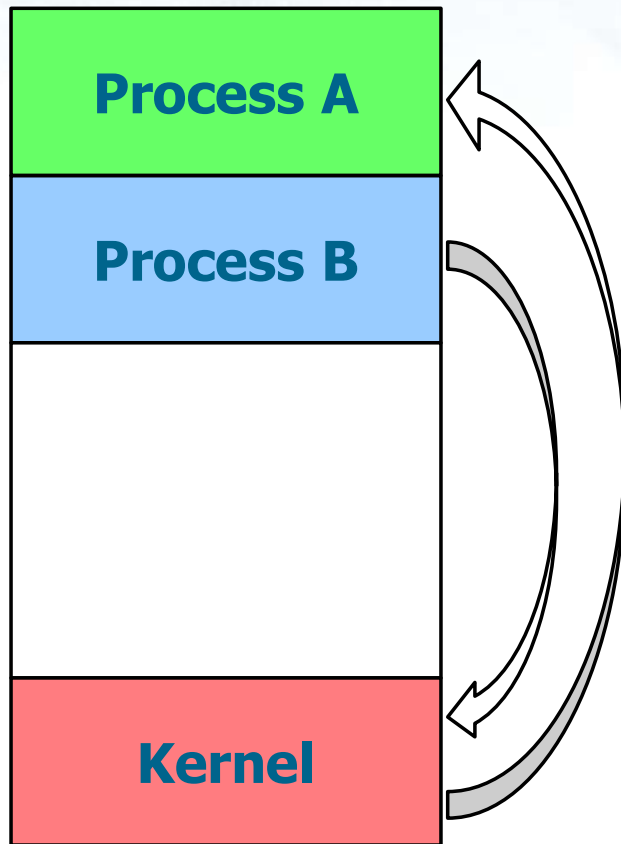
Unit 07
Section 05

▪ Memory sharing

- A chunk of memory is mapped into the address process space

Unit 08
Section 04

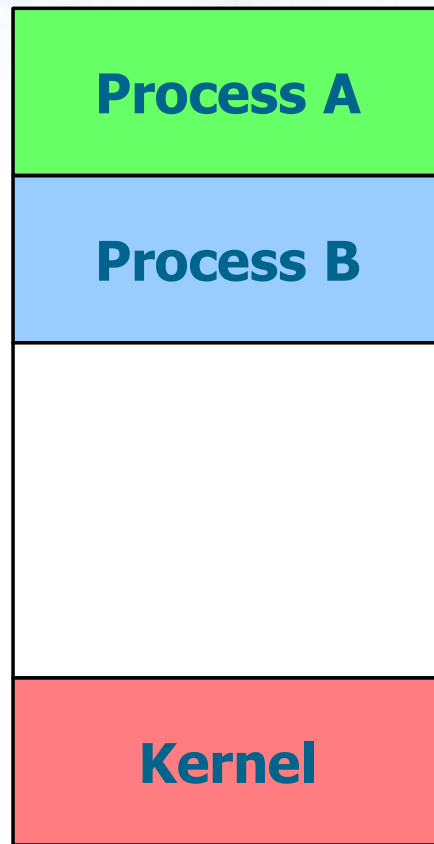
Communication models



❖ Message exchange

- Communication takes place through the exchange of messages
- Need to setup of a communication channel
- Useful for exchanging limited amounts of data
- Uses system calls
 - Require kernel intervention
 - Introduce overhead

Communication models



❖ Message exchange

➤ Most common methods

- Pipes
 - Basic (older) strategy
- FIFOs
 - Avoid a common ancestor between processes
- Message queues
 - Allow structured data
- Network sockets
 - Allow transfer (through the network) between processes running on different machines

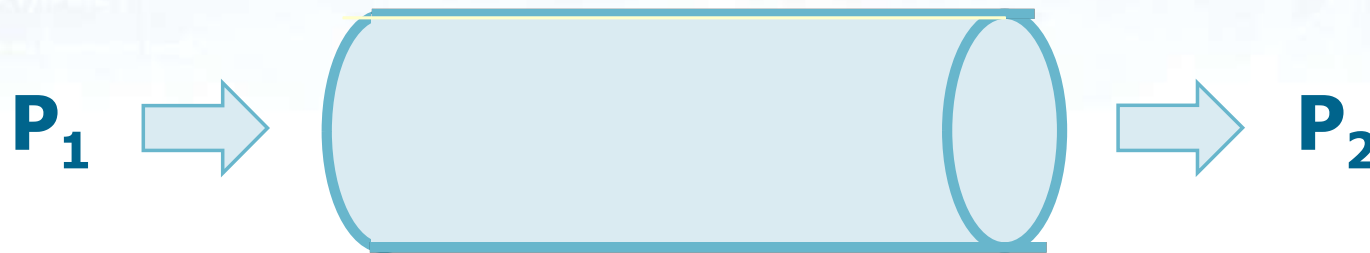
Unit 08
Section 02

Unit 08
Section 03

Unit 08
Section 05

Inter
machine

Review: Pipes

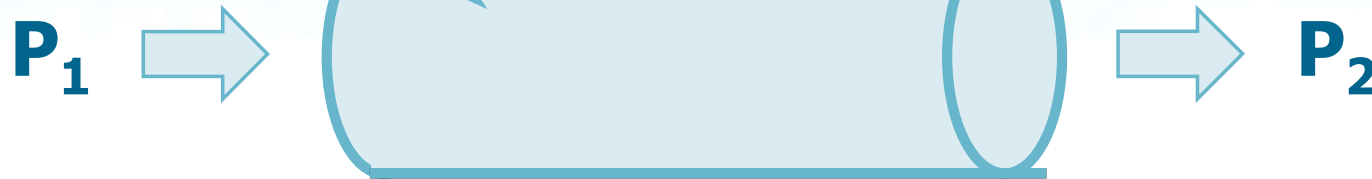


❖ Original pipes (or **unnamed** pipes)

- Are a common form of UNIX System IPC
- Are pseudo-files linking two processes
- Can be used only between processes that have a **common ancestor**
 - The pipe must be created **before** forking the parent process
 - The pipe lasts only as long as the processes last

Review: Pipes

It often has a capacity limited to 64KBytes



❖ Original pipes (or **unnamed** pipes)

➤ Are **half duplex**

- Data flows only in one direction

➤ Can be seen as an **unstructured** sequential files

- The communication channel is a FIFO queue
- Structured data transfers require a communication protocol

➤ Transfer only **limited quantity** of memory



Review: Pipes

```
int file[2];
char cR cW;

if (pipe(file) != 0) { ... Error ... }
pid = fork ();
if (pid == -1) { ... Error ... }

if (pid == 0) {
    // Child
    close (file[1]);
    n = read (file[0], &cR, sizeof(char));
    ...
} else {
    // Parent
    close (file[0]);
    n = write (file[1], &cW, sizeof(char));
    ...
}
```

The child reads
from the pipe

The parent writes
into the pipe